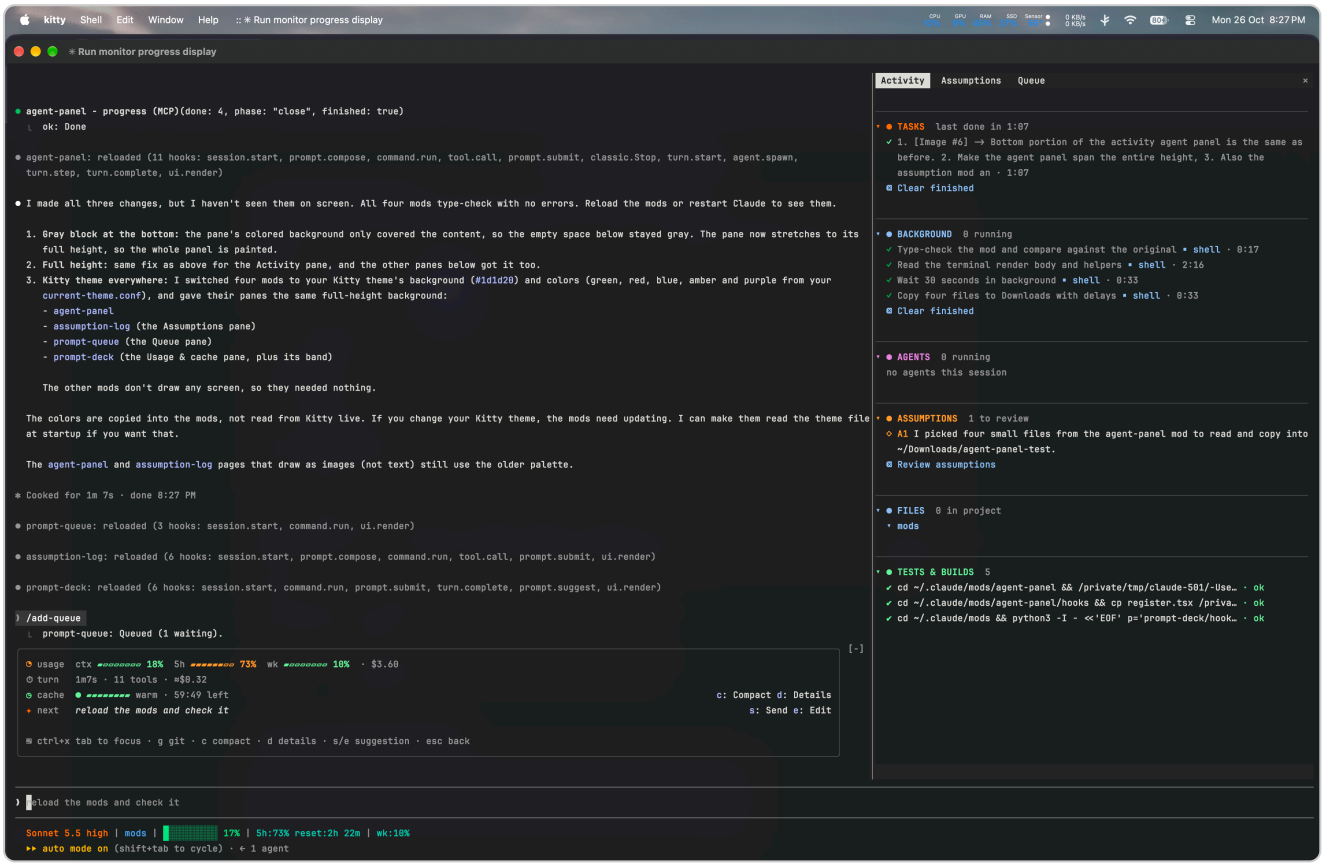
claude-workflow

How to copy Abrar118's Claude Code setup exactly · github.com/Abrar118/claude-workflow



The full setup: chat on the left, agent-panel on the right, prompt-deck band above the prompt, status line at the bottom.

1. What you get

Part	What it is
11 mods	Claude Code plugins that add panes, bands and guards. Built by me.
verify-outside-feedback	A skill that checks pasted feedback claim by claim. Built by me.
Status line	Two-line status bar script.
CLAUDE.md	My global instructions for Claude.
Third-party plugins	superpowers, frontend-design, ui-ux-pro-max and others, installed from their own sources.

2. Before you start

- Claude Code (terminal CLI), signed in.
- `git`, `jq` (status line) and optionally `gh` (PR badge in the status line).
- Python 3 for ui-ux-pro-max's search script.
- Optional, for the same look: the **Kitty** terminal with a dark theme whose background is `#1d1d20`.

3. Install everything (recommended)

```
git clone https://github.com/Abrar118/claude-workflow.git
cd claude-workflow
./install.sh
```

The script adds the marketplaces, installs every mod, my skill and the third-party plugins, clones Uncodixfy, copies the status line, and copies `CLAUDE.md` only if you don't have one yet.

Then open `~/.claude/settings.json` and add the status line block:

```
"statusLine": { "type": "command", "command": "sh ~/.claude/statusline-command.sh" }
```

Restart Claude Code. Settings I also use (optional): `"model": "opus"`, `"effortLevel": "high"`, `"tui": "fullscreen"`. They're all in `settings.example.json`.

4. Or install piece by piece

At the Claude Code prompt, for any mod or the skill:

```
/plugin install <name> --marketplace Abrar118/claude-workflow
```

Answer `y` to add the marketplace, then press Enter for the user scope. From a shell: `claude plugin install <name> --marketplace Abrar118/claude-workflow`.

In the desktop app: click + next to the prompt box → Plugins → Add marketplace → `Abrar118/claude-workflow` → Add plugin.

Alternative: run the mods from a folder (how I run them)

Copy the mod folders and point Claude Code at them. Edits then reload live.

```
mkdir -p ~/.claude/mods && cp -R mods/* ~/.claude/mods/
```

In `~/.claude/settings.json`, list each folder, separated by `:`:

```
"env": {
  "CLAUDE_CODE_PLUGIN_DIRS": "~/.claude/mods/agent-panel:~/.claude/mods/assumption-
log:~/.claude/mods/prompt-deck:~/.claude/mods/prompt-queue:~/.claude/mods/turn-
meter:~/.claude/mods/limit-guard:~/.claude/mods/safety-guard:~/.claude/mods/push-
preflight:~/.claude/mods/format-on-edit:~/.claude/mods/venv-auto:~/.claude/mods/debug-
on-paste",
  "CLAUDE_CODE_PLUGIN_DIR_WATCH": "1"
}
```

> Use one method or the other, not both, or each mod loads twice.

5. The mods

agent-panel

A side pane with tasks, background jobs, agents, assumptions and a live file tree, plus a progress bar above the prompt. Claude reports its plan through the `progress` tool, and subagents through `step`. Open it with `/subagents`.



Three agents running: model, effort, context use, current command.



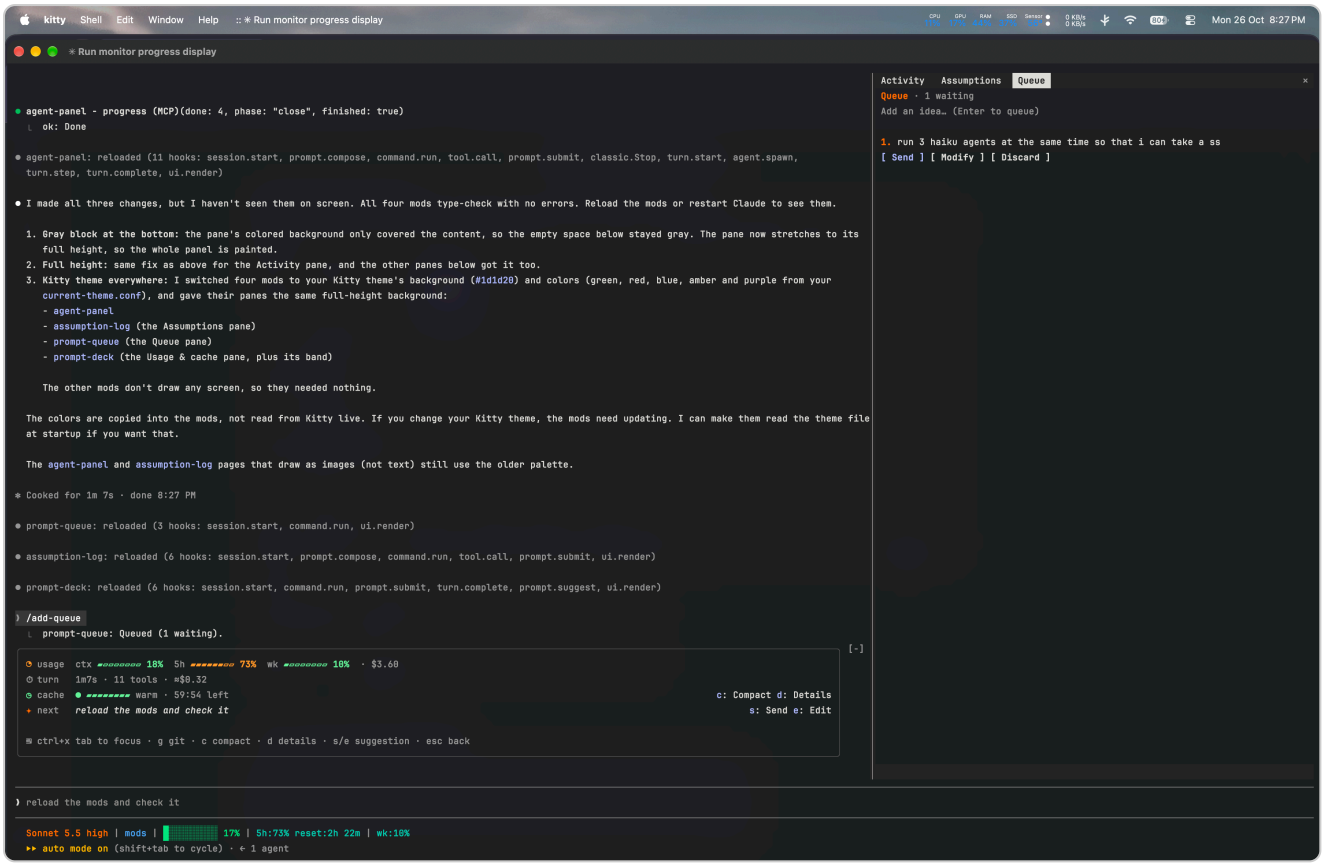
assumption-log: mark each assumption Right, Wrong or Change.

assumption-log

Claude calls an `assume` tool whenever it picks something you didn't say. Open the pane with `/assumptions`, then mark each one right or wrong, or ask for a change. The verdict goes back to Claude.

prompt-queue

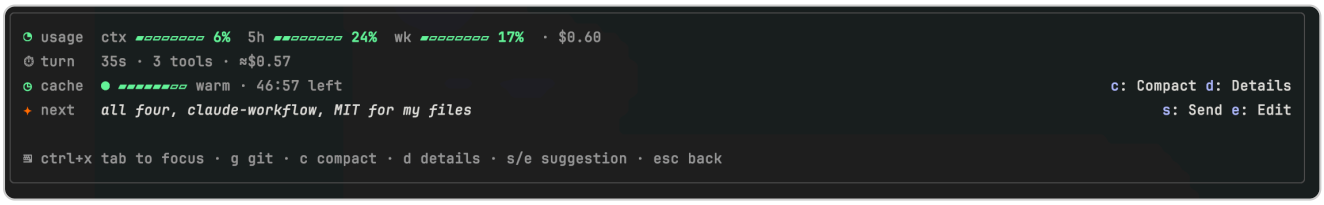
`/add-queue <idea>` saves a prompt while Claude works. `/queue` opens the pane, where you can Send, Modify or Discard each one.



The Queue tab, with one waiting idea.

prompt-deck + turn-meter

A band above the prompt showing context, 5-hour and weekly usage, cost, how long the prompt cache stays warm, the last turn's time and cost, and a suggested next prompt (`s` to send, `e` to edit). `/limits` opens the details. turn-meter measures each turn and prompt-deck shows it, so install both. In the config menu you can set the cache length to 60 or 5 minutes.



The prompt-deck band.

Guards and helpers (no screen)

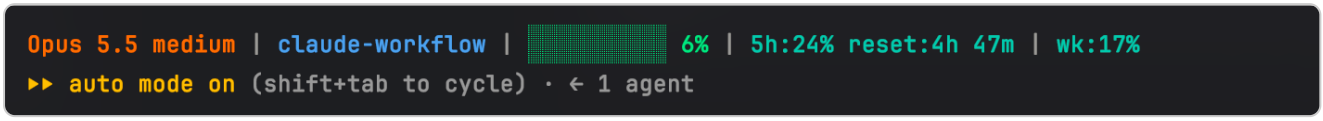
Mod	What it does
limit-guard	Warns as the 5-hour limit fills up; asks before starting subagents when it's nearly used up.
safety-guard	Asks before force pushes, pushes to main, `rm -rf`, hard resets, `DROP` / `TRUNCATE`, `flyway clean` and `.env` edits.
push-preflight	Runs `format:check`, `lint` and `typecheck` before `git push`, and blocks the push if one fails.
format-on-edit	Runs the project's own formatter (Prettier, dart format, Spotless) after each edit.

venv-auto	Activates the nearest `.venv` for python, pip and pytest.
debug-on-paste	Pasted error or stack trace → Claude debugs step by step.

Matching the colors

The panes use my Kitty theme's colors (background `#1d1d20`). They're constants near the top of each mod's `hooks/register.tsx`; edit them to match your terminal.

6. Status line



Model and effort · repo · context bar · 5h usage and reset · weekly usage (plus branch and open PR when there is one).

```
cp statusline/statusline-command.sh ~/.claude/
```

7. My CLAUDE.md

Put this at `~/.claude/CLAUDE.md` (or merge it into yours):

```
# Global Instructions

## Tools
- **Use ripgrep (`rg`) instead of `grep`** for all search operations in Bash commands. It's installed and preferred.

## Outside feedback
- When I paste a review, critique or suggestions (code, research, plans, anything), use the `verify-outside-feedback` skill: give each claim a Confirmed / Refuted / Unclear verdict with evidence before agreeing or changing anything. Never name or guess the source.

## Working Preferences
- Provide concise, focused responses. Skip non-essential context; keep examples minimal. (Override per-request when you want depth.)
- Don't spawn a subagent for work doable directly in one response. Spawn multiple subagents in one turn when fanning out across items or reading multiple files.
- Never claim things about code you haven't opened. If a specific file is referenced, read it before answering.
- Avoid over-engineering. Only make changes directly requested or clearly necessary — no unrequested features, refactors, abstractions, or defensive code for impossible cases.
- Simplest code that works: first ask if it needs to exist, then reuse what's in the codebase, then stdlib, then native platform features, then installed deps; never add a dependency for a few lines. Read and trace the problem fully before picking the small fix; fix bugs at the root, in the shared function, not per caller. Non-trivial logic leaves one small runnable check. Never simplify away validation, security, data-loss error handling or accessibility.
```

8. Third-party plugins

Plugin / skill	Install
superpowers, frontend-design, security-guidance, claude-md-management, claude-code-setup, remember, github	`/plugin install <name>@claude-plugins-official`

ui-ux-pro-max (nextlevelbuilder)	`/plugin install ui-ux-pro-max --marketplace nextlevelbuilder/ui-ux-pro-max-skill`
design-system (claudekit)	`npm i -g ui-ux-pro-max-cli && uipro init --global`
Uncodixfy (cyxzdev)	`git clone https://github.com/cyxzdev/Uncodixfy ~/.claude/skills/uncodixfy`

These belong to their authors and are under their own licenses.

9. My workflow

1. **Idea:** describe the goal; `superpowers:brainstorming` asks questions until it's clear.
2. **Plan:** numbered tasks appear in agent-panel with a progress bar.
3. **Build:** test-first; assumption-log captures every guess for review.
4. **Queue:** new ideas go to prompt-queue instead of interrupting.
5. **Debug:** paste the error; debug-on-paste starts systematic debugging.
6. **Review:** `/code-review`; outside feedback goes through verify-outside-feedback.
7. **Ship:** format-on-edit, push-preflight and safety-guard keep the push clean.

10. Troubleshooting

- **A mod doesn't appear:** run `claude plugin list`, then `/reload-plugins` or restart.
- **Mods load twice:** you installed from the marketplace *and* set `CLAUDE_CODE_PLUGIN_DIRS`. Remove one.
- **Status line is blank:** install `jq`; check the path in `settings.json`.
- **See why a hook failed:** start with `claude --debug`.